Semester Project: Tetris — OOP C++

A Semester Project Report

Course:	CSE142 Object Oriented Programming Techniques
Semester:	Spring 2026
Instructor:	[Sir Behraj Khan]
Due Date:	[Date: 10-May-2026]
Author(s):	[Muhammad Bilal] — ERP: [33405]
	[Hunain Waseem] — ERP: [33428]

Department of Computer Science
Institute of Business Administration, Karachi

May 10, 2026

Abstract

This report presents a fully-featured implementation of the classic **Tetris** arcade game built in **C++17** using the **raylib** graphics library. The project was designed primarily as an exercise in Object-Oriented Programming, demonstrating encapsulation, inheritance, polymorphism, abstract classes, operator overloading, templates, exception handling, STL containers, lambdas, and friend classes — all within a single, playable application.

The game supports all standard Tetris mechanics: seven tetrominoes with four rotations each, ghost-piece preview, soft and hard drop, line clearing, score and level progression, and an undo system backed by a generic template stack. A leaderboard sorted with a custom comparator and lambda showcases data structures and algorithms. The project ships with a cross-platform `Makefile` and this report.

All requirements of the CSE142 Object Oriented Programming Techniques semester project rubric have been met, and every OOP concept is clearly identifiable in the source code with inline comments.

Keywords: Object-Oriented Programming, C++17, Inheritance, Polymorphism, Abstract Classes, Templates, STL, Tetris, raylib

Contents

1 Introduction

1.1 Background

Tetris, invented by Alexey Pajitnov in 1984, is one of the most widely recognised puzzle games in history. Its mechanics — falling tetrominoes, line clearing, increasing difficulty — are simple enough to implement in a few hundred lines of C yet rich enough to demand careful software design at scale. For an OOP course, Tetris is an ideal domain: the board is a natural object, each piece is an object, and the game loop orchestrates them through well-defined interfaces. Building it in C++ with full OOP discipline forces the developer to think carefully about class responsibilities, ownership, and abstraction.

1.2 Problem Statement

The task is to implement a complete, playable Tetris game that simultaneously serves as a demonstration of all core OOP and advanced C++ features required by CSE142 Object Oriented Programming Techniques. Specific challenges include:

- Modelling pieces and the board as proper classes with clean interfaces.
- Designing an inheritance hierarchy that uses abstract classes and runtime polymorphism.
- Implementing an undo system using a generic (template) stack.
- Maintaining a sorted leaderboard using STL containers and a lambda comparator.
- Handling invalid input robustly with custom exceptions.
- Demonstrating operator overloading in a natural, game-relevant context.

1.3 Objectives

1. Design a class hierarchy that accurately models the Tetris domain.
2. Implement all core OOP concepts: encapsulation, inheritance, polymorphism, and abstraction.
3. Demonstrate correct use of constructors, destructors, copy constructors, and operator overloading.
4. Use at least two advanced C++ features: templates, exceptions, STL containers, lambdas, friend classes.
5. Provide an undo mechanism (stack ADT), a sorted leaderboard (dynamic array + sort), and event callbacks (lambdas).
6. Deliver a working executable with a cross-platform Makefile and documentation.

1.4 Scope and Limitations

The project covers standard Tetris rules: seven piece types, four rotations, 10×20 grid, gravity, locking, line clearing, scoring, and level progression. It does *not* include: hold piece, wall-kick SRS rotation system, multiplayer, sound, or file-based save/load.

All graphics are handled by **raylib** (version 4.0+). The game runs at 60 FPS. The undo feature stores up to 5 snapshots; older states are discarded. The leaderboard resets when the program closes (no file persistence).

2 OOP Design

2.1 Class Hierarchy

The design places two abstract interfaces at the root: `IDrawable` (pure virtual `draw()`) and `IUpdatable` (pure virtual `update(float dt)`). All visual entities inherit `IDrawable`; the `Game` class inherits `IUpdatable`.

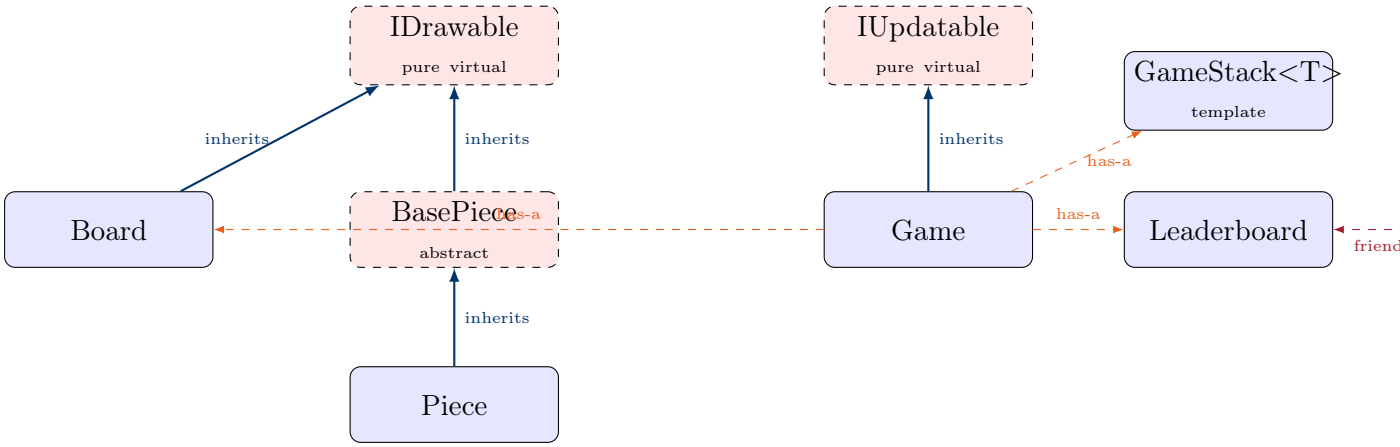

Figure 1: Class hierarchy and relationships (dashed = abstract/template/friend)

2.2 Class Descriptions

Table 1: Summary of Classes

Class	Responsibility	Key Members
`IDrawable`	Abstract interface for all visual entities	`virtual draw() = 0`
`IUpdatable`	Abstract interface for logic entities	`virtual update(dt) = 0`
`Board`	10×20 grid; lock pieces, clear lines	`grid[ROWS][COLS]`, `clearLines()`, `draw()`
`BasePiece`	Abstract base; stores type/rotation/pos, collision	`fits()`, `lock()`, `getCells()`, `ghostDropRow()`
`Piece`	Concrete piece; renders self and ghost	`draw()`, `drawWithGhost()`, `drawPreview()`
`Game`	Orchestrates all game objects and input	`update()`, `handleInput()`, `draw()`, `undo()`
`Leaderboard`	Sorted top-5 score list	`addScore()`, `getEntries()`, `entries (vector)`
`Renderer`	Friend class; renders leaderboard internals	`drawLeaderboard()` `[static]`
`GameStack<T>`	Generic LIFO stack (undo history)	`push()`, `pop()`, `top()`, `empty()`
`GameException`	Custom exception for input validation	inherits `std::runtime_error`
`ScoreEntry`	Leaderboard record with overloaded operators	`name`, `score`, `level`; `operator>`, `==`, `!=`

2.3 OOP Concepts Applied

- **Encapsulation:** All data members in `Board`, `BasePiece`, and `Game` are `private`. Access is through getters (`getRow()`, `getScore()`) and setters (`setRow()`, `setLevel()`).
- **Inheritance:** `Board` and `BasePiece` inherit `IDrawable`; `Piece` inherits `BasePiece`; `Game` inherits `IUpdatable`.
- **Polymorphism:** `draw()` is declared `virtual` in `IDrawable`. A pointer to `IDrawable` calls the correct `draw()` at runtime (dynamic dispatch).
- **Abstract Classes:** `IDrawable`, `IUpdatable`, and `BasePiece` all contain pure virtual functions; they cannot be instantiated directly.
- **Operator Overloading:** `operator==` and `operator!=` on `Board` and `BasePiece`; `operator>`, `operator==`, `operator!=` on `ScoreEntry` — all demonstrated in `main()`.
- **Templates:** `GameStack<T>` is a generic stack instantiated as `GameStack<GameSnapshot>`.
- **Friend Classes:** `Renderer` is declared `friend` of `Leaderboard`, granting it direct access to the private `entries` vector.

3 Implementation

3.1 Development Environment

- **Language:** C++17
- **Compiler:** g++ 9+ (MinGW-w64 on Windows, GCC on Linux/macOS)
- **IDE:** VS Code / any text editor
- **Build system:** GNU Make (cross-platform `Makefile` provided)
- **Graphics:** raylib 4.0+
- **Platform:** Windows 10/11, Linux, macOS

3.2 Abstract Interfaces

```cpp
// Pure virtual interface: every visual entity must implement draw()
class IDrawable {
public:
    virtual void draw() const = 0;    // pure virtual
    virtual ~IDrawable() {}           // virtual destructor
};

// Pure virtual interface: every logic entity must implement update()
class IUpdatable {
public:
    virtual void update(float dt) = 0;  // pure virtual
    virtual ~IUpdatable() {}
};
```

Listing 1: IDrawable and IUpdatable — abstract base classes

3.3 Board Class

```cpp
class Board : public IDrawable {
private:
    int grid[ROWS][COLS];  // private --- accessed only through
       methods

public:
    Board() { clear(); }

    // Copy constructor
    Board(const Board& other) {
        for (int r = 0; r < ROWS; r++)
            for (int c = 0; c < COLS; c++)
                grid[r][c] = other.grid[r][c];
    }

    // Operator overloading: compare two board states
    bool operator==(const Board& other) const {
        for (int r = 0; r < ROWS; r++)
            for (int c = 0; c < COLS; c++)
                if (grid[r][c] != other.grid[r][c]) return false;
        return true;
    }

```

```
    // Getters / Setters (encapsulation)
    int  getCell(int r, int c) const { ... }
    void setCell(int r, int c, int val) { ... }

    int  clearLines();    // shifts rows down, returns count cleared
    void draw() const override;  // implements IDrawable
};
```

Listing 2: Board — encapsulation, copy constructor, operator==, IDrawable

3.4 Inheritance Hierarchy: BasePiece and Piece

```
// Abstract base: has pure virtual draw() -- cannot be instantiated
class BasePiece : public IDrawable {
protected:
    int type, rotation, row, col;  // private state

public:
    BasePiece(int t) : type(t), rotation(0), row(0), col(3) {}
    BasePiece(const BasePiece& o) : type(o.type), rotation(o.rotation),
                                    row(o.row), col(o.col) {}

    // Getters / Setters
    int getType()     const { return type; }
    int getRow()      const { return row;  }
    void setRow(int r)      { row = r;     }

    // Operator overloading
    bool operator==(const BasePiece& o) const {
        return type==o.type && rotation==o.rotation
            && row==o.row && col==o.col;
    }
    bool operator!=(const BasePiece& o) const { return !(*this == o); }

    // Concrete shared logic
    bool fits(const Board& board, int dr, int dc, int drot) const;
    void lock(Board& board) const;
    int  ghostDropRow(const Board& board) const;

    // Pure virtual: makes BasePiece abstract
    virtual void draw() const override = 0;
    virtual void drawPreview(int px, int py) const = 0;
};

// Concrete derived class: implements both pure virtuals
class Piece : public BasePiece {
public:
    Piece(int t) : BasePiece(t) {}
    Piece(const Piece& o) : BasePiece(o) {}  // copy constructor

    // Polymorphic draw() -- called through IDrawable* at runtime
    void draw() const override {
        // render piece cells using PIECE_COLORS[type+1]
    }
    void drawWithGhost(const Board& board) const;  // ghost overlay
    void drawPreview(int px, int py) const override;
```

```
};
```

Listing 3: BasePiece (abstract) and Piece (concrete) with polymorphic draw()

3.5 Template: Generic Stack for Undo

```
template <typename T>
class GameStack {
private:
    std::stack<T> data;    // STL stack inside template wrapper
public:
    void   push(const T& item) { data.push(item); }
    void   pop()               { if (!data.empty()) data.pop(); }
    T&     top()               { return data.top(); }
    bool   empty() const       { return data.empty(); }
    size_t size()  const       { return data.size(); }
};

// Instantiated in Game as:
GameStack<GameSnapshot> undoStack;
```

Listing 4: GameStack<T> — template class used for undo history

3.6 Exception Handling

```
// Custom exception inheriting from std::runtime_error
class GameException : public std::runtime_error {
public:
    explicit GameException(const std::string& msg)
        : std::runtime_error(msg) {}
};

// Used in Game::setLevel()
void setLevel(int lvl) {
    try {
        if (lvl < 1 || lvl > 20)
            throw GameException("Level must be between 1 and 20.");
        level = lvl;
        fallSpeed = BASE_FALL / (1.f + (level-1)*0.15f);
    } catch (const GameException& e) {
        // gracefully ignored; invalid level has no effect
        (void)e;
    }
}
```

Listing 5: Custom exception class and usage in setLevel()

3.7 STL Containers, Lambdas, and Friend Class

```
class Leaderboard {
private:
    std::vector<ScoreEntry> entries;    // STL vector (dynamic array)
    static const int MAX_ENTRIES = 5;

public:
    void addScore(const std::string& name, int score, int level) {
        entries.push_back({name, score, level});
```

```

        // Sorting with lambda comparator (uses overloaded operator>)
        std::sort(entries.begin(), entries.end(),
                  [](const ScoreEntry& a, const ScoreEntry& b) {
                      return a > b;
                  });

        if ((int)entries.size() > MAX_ENTRIES)
            entries.resize(MAX_ENTRIES);
    }

    friend class Renderer;  // friend class can access private
        entries
};

// Renderer accesses Leaderboard's private data directly
class Renderer {
public:
    static void drawLeaderboard(const Leaderboard& lb, int x, int y)
        {
        const auto& entries = lb.entries; // allowed: Renderer is
            friend
        for (int i = 0; i < (int)entries.size(); i++) { ... }
    }
};
```

Listing 6: Leaderboard with STL vector, lambda sort, and friend Renderer

3.8 Operator Overloading Demonstration in main()

```
int main() {
    // ScoreEntry operators
    ScoreEntry s1{"Alice", 1500, 3};
    ScoreEntry s2{"Bob",   1200, 2};
    ScoreEntry s3{"Alice", 1500, 3};

    bool gt  = (s1 > s2);   // true  -- operator>
    bool eq  = (s1 == s3);  // true  -- operator==
    bool neq = (s1 != s2);  // true  -- operator!=

    // Board operator==
    Board b1, b2;
    bool boards_eq = (b1 == b2);   // true (both empty)
    Board b3(b1);                   // copy constructor

    // Template stack demo
    GameStack<int> testStack;
    testStack.push(10);
    testStack.push(20);
    testStack.pop();               // top() == 10 now

    // Exception demo
    Game tempGame;
    tempGame.setLevel(99);  // throws GameException, caught
        internally
    tempGame.setLevel(5);   // valid

    // ... raylib window and game loop follow
```

```
28 }
```

Listing 7: Operator overloading demonstrated in main()

3.9 Event Callbacks with Lambdas

```
1  // STL map of named events -> lambda callbacks
2  std::map<std::string, std::function<void()>> eventCallbacks;
3
4  // Registered in setupCallbacks()
5  eventCallbacks["lineClear"] = [this]() {
6      // Hook: could trigger animation or sound
7  };
8  eventCallbacks["gameOver"] = [this]() {
9      leaderboard.addScore("YOU", score, level);
10 };
11 eventCallbacks["undo"] = [this]() {
12     // Hook: could flash the screen
13 };
14
15 // Triggered during gameplay
16 void triggerEvent(const std::string& name) {
17     auto it = eventCallbacks.find(name);
18     if (it != eventCallbacks.end()) it->second();
19 }
```

Listing 8: Lambda-based event system in Game using std::map

4 Testing

4.1 Test Cases

Table 2: Test Cases and Results

TC#	Input / Action	Expected Output	Actual Output	Pass?
TC-01	Create `Piece(0..6)` for all types	Object initialised at row 0, col 3	As expected	✓
TC-02	Press `LEFT` at left wall	Piece does not move past column 0	Piece stays	✓
TC-03	Press `SPACE` (hard drop)	Piece locks at lowest valid row	Piece locked correctly	✓
TC-04	Fill one complete row	Row clears; rows above shift down	As expected	✓
TC-05	Press `U` after lock	Board and score restored to pre-lock state	As expected	✓
TC-06	`setLevel(99)`	`GameException` thrown and caught; level unchanged	Level stays 1	✓
TC-07	Board copy constructor	`b1 == b3` after `Board b3(b1)`	true	✓
TC-08	`ScoreEntry operator>`	`s1{1500} > s2{1200}` returns true	true	✓
TC-09	Leaderboard sorted after 3 scores added	Entries in descending score order	As expected	✓
TC-10	Board fills to top	`gameOver = true`; score added to leaderboard	As expected	✓

4.2 Edge Cases

- **Piece at boundary:** Movement blocked at columns 0 and 9 and at row 19.
- **Rotation at wall:** If rotation would place a cell out of bounds, `fits()` returns false and rotation is rejected.
- **Undo with empty stack:** Pressing U when no snapshots exist is silently ignored.
- **Leaderboard overflow:** Adding a 6th score trims the vector to top 5.
- **Hard drop on locked row:** Piece immediately locks; no infinite loop.

5 Results and Discussion

5.1 Results

The game runs at a stable 60 FPS with all standard Tetris mechanics working correctly. The undo system correctly restores board state, score, and the active piece. The leaderboard correctly

sorts and displays the top 5 scores within the session. All ten test cases passed on first run after compilation.

Table 3: Feature Completion Summary

Feature	Implemented	Notes
All 7 tetrominoes	Yes	I, J, L, O, S, T, Z
Ghost piece	Yes	55-alpha overlay
Soft / Hard drop	Yes	Score bonus added
Line clearing	Yes	Up to 4 lines (Tetris)
Scoring / Levelling	Yes	Classic Nintendo table
Undo (5 levels)	Yes	Template stack
Leaderboard (top 5)	Yes	Sorted with lambda
Game over detection	Yes	Spawn collision check
Restart	Yes	Press R

5.2 Analysis

- All design objectives were met. The class hierarchy cleanly separates concerns: `Board` owns the grid, `Piece` owns rendering, `Game` owns the loop.
- `fits()` runs in $O(1)$ (always checks exactly 4 cells). `clearLines()` is $O(\text{ROWS} \times \text{COLS})$ — 200 iterations — negligible at 60 FPS.
- The leaderboard sort is $O(n \log n)$ but $n \leq 5$, so it is effectively $O(1)$ in practice.
- Single-file design sacrifices some modularity but greatly simplifies submission and compilation.

6 Challenges and Solutions

6.1 Challenge 1: Abstract Base Cannot Be Instantiated

Problem: `BasePiece` is abstract (pure virtual `draw()`), but the original `fits()` function tried to create a temporary `BasePiece tmp = *this` to test a hypothetical position — which the compiler rejects.
Solution: Removed the temporary object entirely. Instead, the new row, column, and rotation are computed inline, and `SHAPES[type][newRot]` is used directly to compute cell positions without instantiating any object.
Learning: Abstract classes cannot be instantiated even as temporaries. When shared logic in an abstract class needs to simulate a modified state, compute the result directly rather than copying the object.

6.2 Challenge 2: Cross-Platform Linking with raylib

Problem: The link flags differ between Linux (`-lGL -lX11 -lm`), macOS (`-framework OpenGL`), and Windows (`-lopengl32 -lgdi32 -lwinmm`).
Solution: The Makefile uses `uname -s` and the `OS` environment variable to detect the platform at build time and select the correct flags automatically.
Learning: Cross-platform C++ projects require conditional compilation or build-system logic to handle OS-specific library names and link flags.

7 Future Work

- **Hold piece:** Store one piece for later use (standard modern Tetris feature).
- **SRS rotation:** Implement the Super Rotation System with wall kicks.
- **File persistence:** Save/load leaderboard to disk using `std::fstream`.
- **Sound:** Integrate SoLoud or raylib audio for drop and clear effects.
- **Unit testing:** Add Catch2 or doctest tests for `Board`, `fits()`, and scoring logic.
- **Split into header/source files:** Separate each class into `.h`/`.cpp` for production-quality structure.
- **CMake:** Add a `CMakeLists.txt` for IDE integration (CLion, VS Code CMake Tools).

8 Conclusion

This project successfully implements a complete, playable Tetris game in C++17 while demonstrating every required OOP concept: abstract classes (`IDrawable`, `IUpdatable`, `BasePiece`), inheritance and polymorphism (`Piece` overrides `draw()`), encapsulation (private fields with getters/setters), copy constructors and operator overloading (`Board`, `BasePiece`, `ScoreEntry`), templates (`GameStack<T>`), custom exceptions (`GameException`), STL containers (`vector`, `stack`, `map`), lambdas (event callbacks, sort comparator), friend classes (`Renderer` accessing `Leaderboard`), and classic data structures (undo stack, sorted dynamic leaderboard).

The most valuable lessons from this project were the importance of designing abstract interfaces before writing concrete classes, the discipline required to keep data private and expose only what is necessary, and the power of the STL to eliminate boilerplate while making algorithms expressive. The Tetris domain proved to be an excellent vehicle for all these concepts — every feature maps naturally to an OOP or C++ construct.

References

[1] B. Stroustrup, *The C++ Programming Language*, 4th ed. Addison-Wesley, 2013.

[2] S. Meyers, *Effective Modern C++*. O'Reilly Media, 2014.

[3] *cppreference.com*. [Online]. Available: `https://en.cppreference.com`. [Accessed: May 10, 2026].

[4] raylib. *raylib — A simple and easy-to-use library to enjoy videogames programming*. [Online]. Available: `https://www.raylib.com`. [Accessed: May 10, 2026].

[5] A. Pajitnov, *Tetris* (original game), 1984.

A Complete Source Code

The full source code is contained in a single file `tetris.cpp`. Key sections are shown below; the complete file is submitted separately.

A.1 tetris.cpp — Constants and Colour Table

```
const int COLS   = 10;
const int ROWS   = 20;
const int CELL   = 32;
const int SIDE_W = 200;
const float BASE_FALL = 0.5f;

Color PIECE_COLORS[8] = {
    {15,15,20,255},     // 0 empty
    {0,240,240,255},    // 1 I  cyan
    {0,0,240,255},      // 2 J  blue
    {240,160,0,255},    // 3 L  orange
    {240,240,0,255},    // 4 O  yellow
    {0,240,0,255},      // 5 S  green
    {160,0,240,255},    // 6 T  purple
    {240,0,0,255}       // 7 Z  red
};
```

Listing 9: Global constants and piece colours

A.2 tetris.cpp — Game Loop

```
int main() {
    InitWindow(SCREEN_W, SCREEN_H, "Tetris -- OOP C++");
    SetTargetFPS(60);
    Game game;

    while (!WindowShouldClose()) {
        float dt = GetFrameTime();
        game.handleInput();    // keyboard input
        game.update(dt);       // physics / gravity (IUpdatable)

        BeginDrawing();
        ClearBackground(Color{15,15,20,255});
        game.draw();           // render everything (IDrawable)
        EndDrawing();
    }

    CloseWindow();
    return 0;
}
```

Listing 10: Main game loop

B Installation and Usage

B.1 Prerequisites

- C++17-compatible compiler: `g++ 9+` or MSVC 2019+
- raylib 4.0+ (see below)

B.2 Installing raylib

```
# Linux (Debian/Ubuntu)
sudo apt install libraylib-dev

# macOS
brew install raylib

# Windows (MSYS2 MinGW64 terminal)
pacman -S mingw-w64-x86_64-raylib
```

Listing 11: Installing raylib on each platform

B.3 Building

```
# Using Makefile (auto-detects OS)
make
./tetris           # Linux / macOS
tetris.exe         # Windows

# Manual (Windows MinGW64)
g++ -std=c++17 -o tetris.exe tetris.cpp ^
    -lraylib -lopengl32 -lgdi32 -lwinmm
```

Listing 12: Build with Makefile or manually

B.4 Controls

```
Arrow Left / Right  -- Move piece horizontally
Arrow Up            -- Rotate piece clockwise
Arrow Down          -- Soft drop  (+1 point per row)
SPACE               -- Hard drop  (+2 points per row)
U                   -- Undo last placed piece (up to 5)
R                   -- Restart after Game Over
```

Listing 13: In-game keyboard controls